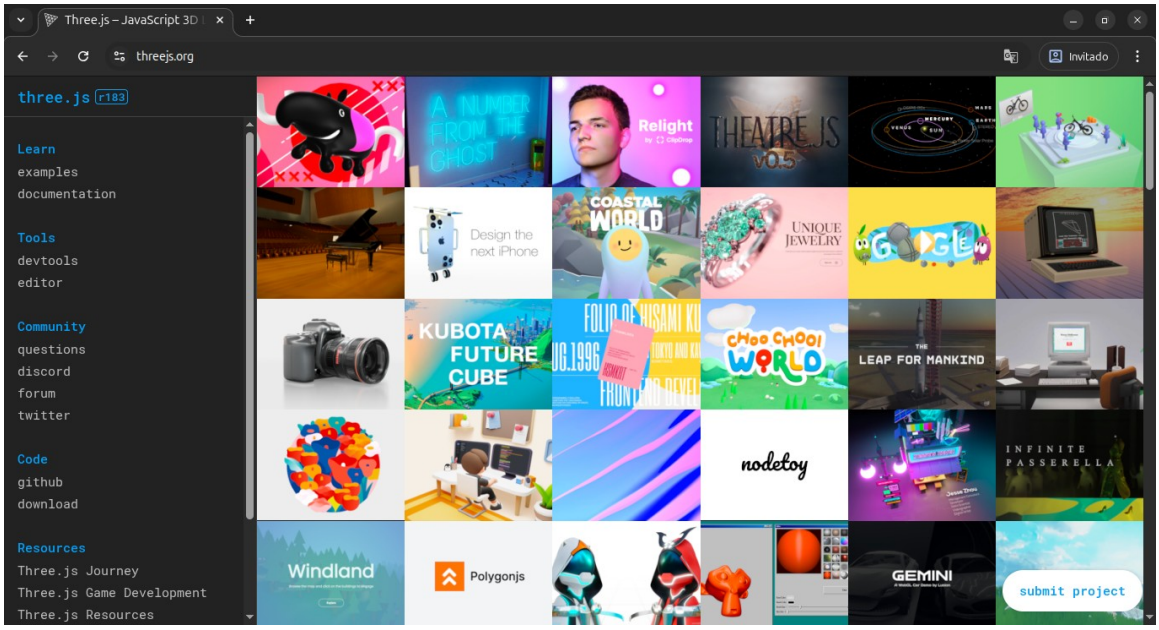
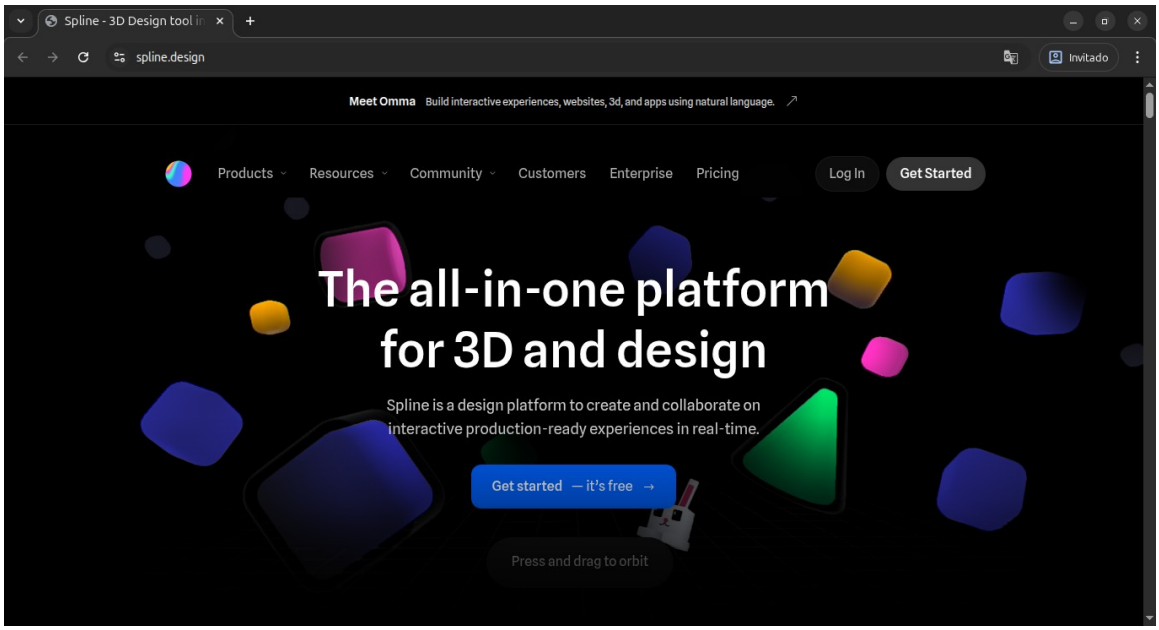
Introducción y Aplicación de Three.js en la Creación de Experiencias 3D



Introducción

Three.js es una biblioteca de JavaScript orientada al desarrollo de gráficos tridimensionales (**3D**) dentro del navegador web. Su principal función es simplificar el uso de **WebGL**, una tecnología de bajo nivel utilizada para renderizar gráficos complejos, permitiendo a los desarrolladores enfocarse en la lógica visual y la interacción.

En el contexto actual del desarrollo web, donde la **experiencia del usuario** es un factor clave, herramientas como Three.js permiten crear entornos **interactivos, dinámicos y visualmente atractivos** sin necesidad de motores gráficos avanzados.

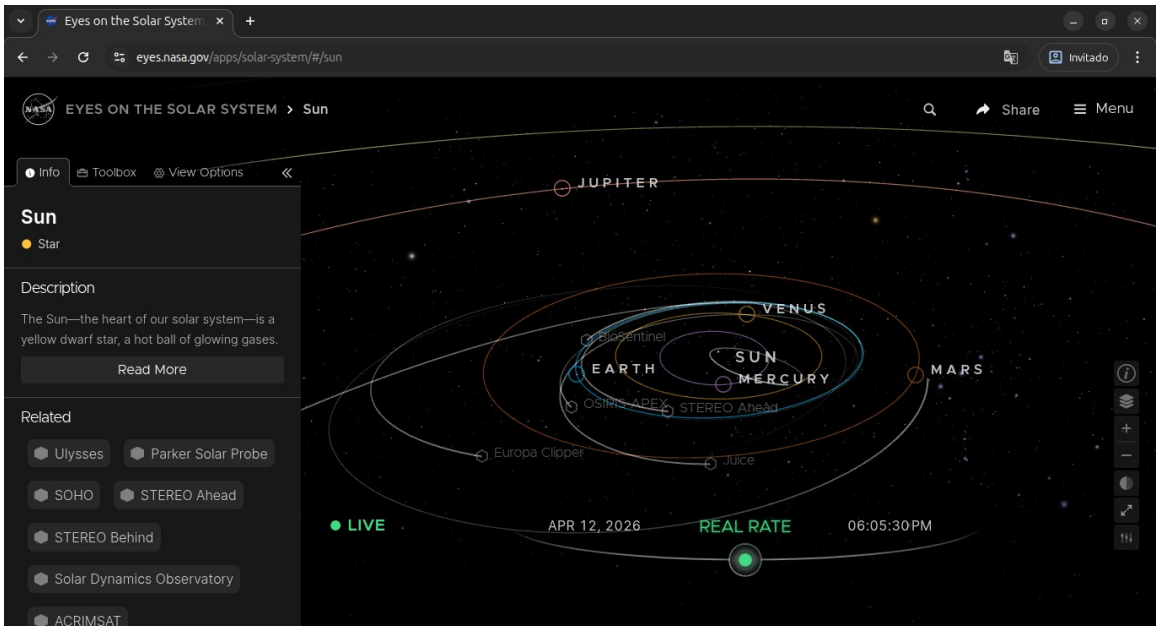


¿Qué es Three.js?

Three.js es una **capa de abstracción** que facilita la creación de gráficos 3D sobre WebGL. En lugar de trabajar directamente con código complejo de renderizado, el desarrollador interactúa con:

- Objetos
- Escenas
- Cámaras

Esto permite construir entornos tridimensionales mediante una **arquitectura clara y modular**.

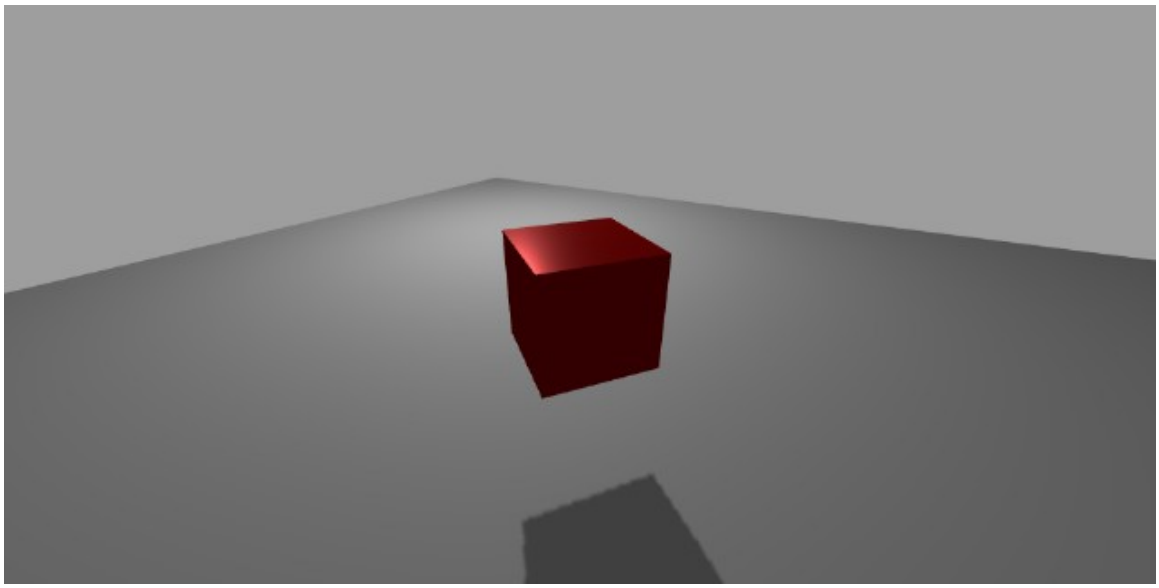


¿Para qué sirve?

Three.js tiene múltiples aplicaciones en el desarrollo web moderno:

- **Visualización de productos** en comercio electrónico
- **Simuladores interactivos**
- **Videojuegos en navegador**
- **Aplicaciones educativas**
- **Arquitectura y recorridos virtuales**
- **Marketing digital y experiencias inmersivas**

Su versatilidad permite adaptarse tanto a proyectos **académicos** como **profesionales**.



¿Cómo funciona Three.js?

El flujo básico de funcionamiento es el siguiente:

1. Creación de una **escena (Scene)**
2. Definición de una **cámara (Camera)**
3. Configuración de un **renderizador (Renderer)**
4. Creación de objetos (**Geometry + Material**)
5. Renderizado en el navegador

Este proceso permite construir progresivamente una experiencia visual completa.

Componentes y comandos básicos de Three.js

- Scene (Escena)

Es el **contenedor principal** donde se agregan todos los elementos del entorno.

```
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x9e9e9e);
```

Propiedades importantes:

- background: color o textura de fondo
- fog: efecto de niebla
- children: objetos dentro de la escena

- Camera (Cámara)

Define el **punto de vista** desde el cual se observa la escena.

```
const camera = new THREE.PerspectiveCamera(75, width /
height, 0.1, 1000);
camera.position.set(-1.5, 1.5, -2.5);
```

Propiedades principales:

- position (x, y, z)
- fov (campo de visión)
- aspect (relación de aspecto)
- near y far

- Renderer (Renderizador)

Se encarga de **dibujar la escena** en el navegador.

```
const renderer = new THREE.WebGLRenderer({ antialias: true });
```

```
renderer.setSize(width, height);
```

```
renderer.setPixelRatio(window.devicePixelRatio);
```

Funciones clave:

- setSize(width, height)
- render(scene, camera)
- domElement

- Geometry (Geometría)

Define la **forma del objeto**.

```
geometry = new THREE.BoxGeometry();
```

Ejemplos:

- BoxGeometry (cubo)
- SphereGeometry (esfera)
- PlaneGeometry (plano)

- Material (Material)

Define la **apariencia visual** del objeto.

```
material = new THREE.MeshStandardMaterial({ color: 0xff0000,
metalness: 0.5, roughness: 0.2 });
```

Propiedades:

- color
- wireframe
- opacity
- transparent

- Mesh (Malla)

Es la combinación de geometría y material.

```
currentMesh = new THREE.Mesh(geometry, material);
```

```
scene.add(currentMesh);
```

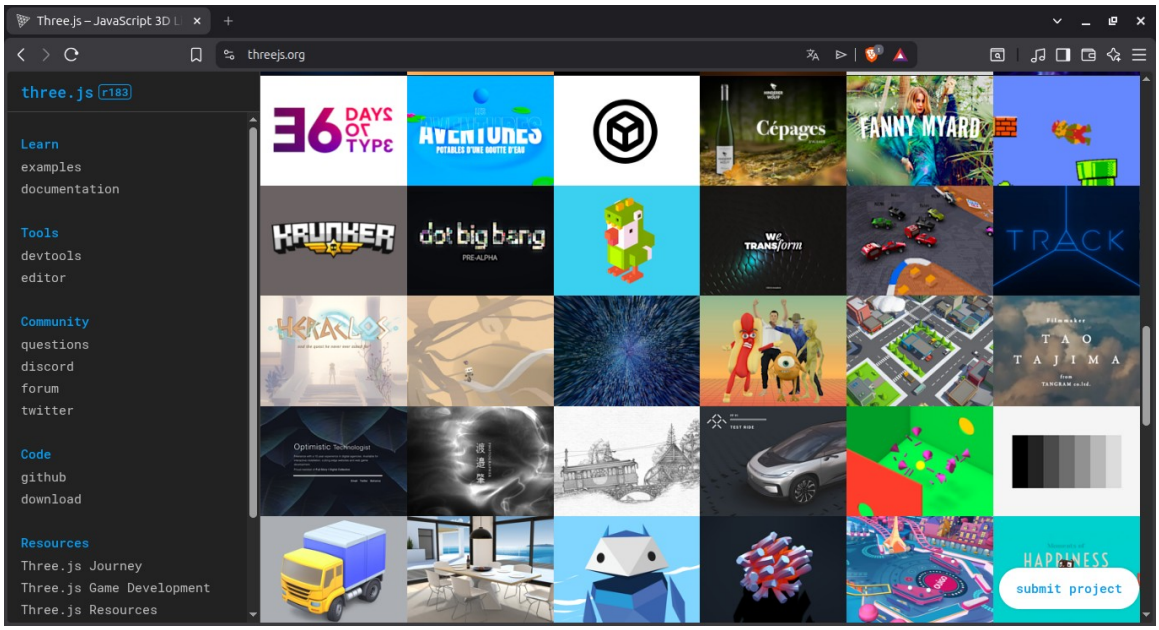
- Light (Iluminación)

Permite visualizar los objetos con **profundidad y realismo**.

```
ambientLight = new THREE.AmbientLight(0xffffff, 0.20);

scene.add(ambientLight);

light = new THREE.PointLight(0xffffff, 50);

light.position.set(5, 5, 5);

scene.add(light);
```

Tipos principales:

- AmbientLight (luz general)
- DirectionalLight (luz direccional)
- PointLight (luz puntual)

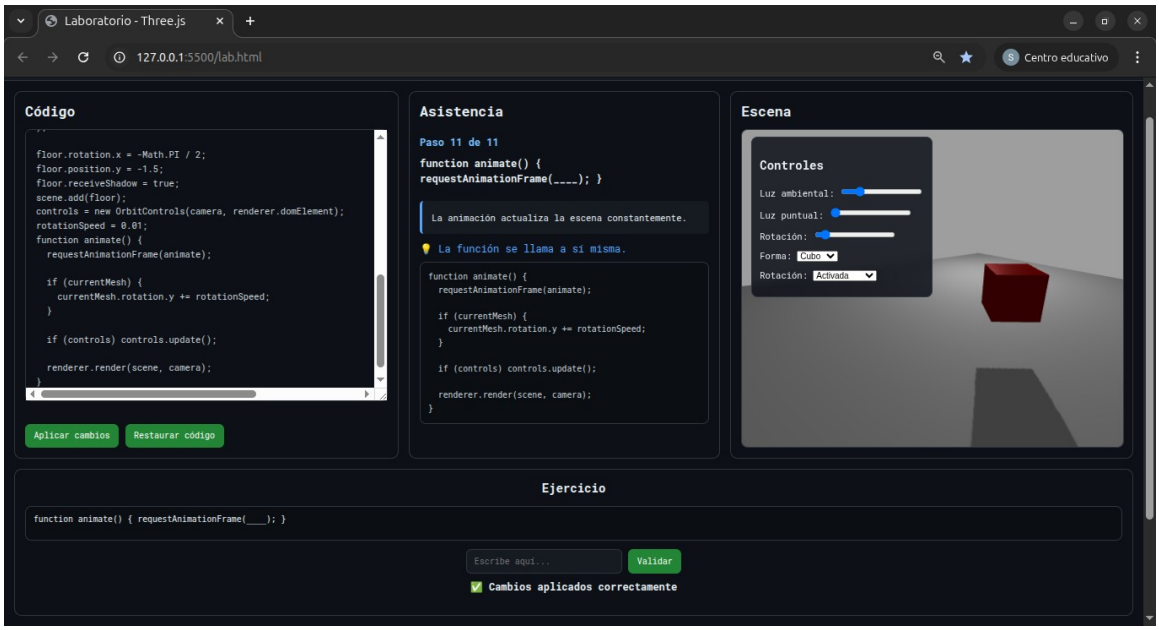


Ventajas de Three.js

- Simplifica el uso de **WebGL**
- Compatible con **navegadores modernos**
- Permite crear experiencias **interactivas**
- Cuenta con una **amplia comunidad**
- Integración con **HTML, CSS y JavaScript**

Recurso oficial

Para más información: https://threejs.org



Aplicación práctica: Laboratorio interactivo

Este proyecto integra un **laboratorio interactivo** que permite aplicar los conceptos aprendidos mediante la construcción progresiva de una escena 3D.

Incluye:

- Sistema por pasos
- Validación de respuestas
- Edición de código
- Renderizado en tiempo real
- Retroalimentación inmediata